

Math 501 -- HW 1 - Steel

1.) Given the IBVPs, do the following

$$\text{I: } U_t = \nu U_{xx}, x \in (0,1), t > 0$$

$$U(x,0) = \sin(2\pi x) \quad x \in [0,1]$$

$$U(0,t) = U(1,t) = 0, t \geq 0$$

$$\text{II: } U_t = U_{xx}, x \in (0,1), t > 0$$

$$U(x,0) = \cos(\pi x/2) \quad x \in [0,1]$$

$$U_x(0,t) = U(1,t) = 0 \quad t \geq 0$$

i.) find exact solutions

$$\square \text{ I: assume } U(x,t) = F(x) G(t)$$

$$\text{then this becomes } \frac{dG}{dt} = -\nu \lambda G \text{ \& } \frac{d^2 F}{dx^2} + \lambda F = 0$$

if $\lambda \leq 0$ then this permits only trivial solutions
so assume $\lambda > 0$

$$\frac{d^2 F}{dx^2} + \lambda F = 0 \text{ has a solution of the}$$

$$\text{form } F(x) = c_1 \cos(\sqrt{\lambda} x) + c_2 \sin(\sqrt{\lambda} x)$$

now, applying BCs

$$F(0) = 0 \Rightarrow c_1 \cos(0) + c_2 \sin(0) = 0 \Rightarrow c_1 = 0$$

$$F(1) = 0 \Rightarrow c_2 \sin(\sqrt{\lambda}) = 0$$

$$\Rightarrow \lambda_n = (n\pi)^2 \quad n \in \mathbb{N}$$

$$\begin{aligned} \frac{dG}{dt} = -\nu \lambda_n G &\Rightarrow G(t) = c_3 e^{-\nu \lambda_n t} \\ &= c_3 e^{-(n\pi)^2 t} \end{aligned}$$

HW 2 - Stell

I. a (cont.) now $u(x,t) = F(x)G(t) = c \sin(n\pi x) e^{-v(n\pi)^2 t}$

$$u(x,0) = \sin(2\pi x) \Rightarrow n=2, c=1$$

so I has form of $u(x,t) = \sin(2\pi x) e^{-v(2\pi)^2 t}$

II: likewise assume $u(x,t) = F(x)G(t)$

which leads to $\frac{dG}{dt} = -\lambda G$ & $\frac{d^2 F}{dx^2} + \lambda F = 0$

assume $\lambda > 0$ otherwise it is trivial solution.

this leads to

$$F(x) = c_1 \cos(\sqrt{\lambda} x) + c_2 \sin(\sqrt{\lambda} x)$$

$$F'(x) = -c_1 \sqrt{\lambda} \sin(\sqrt{\lambda} x) + c_2 \sqrt{\lambda} \cos(\sqrt{\lambda} x)$$

applying Bcs

$$F(0) = 0 \Rightarrow c_2 = 0$$

$$F(1) = 0 \Rightarrow c_1 \cos(\sqrt{\lambda}) = 0 \Rightarrow \lambda_n = \left(n\pi + \frac{\pi}{2}\right)^2 \quad n \in \mathbb{N}$$

$$\Rightarrow F(x) = c_1 \cos\left(\left(n\pi + \frac{\pi}{2}\right)x\right)$$

$$\frac{dG}{dt} = -\lambda G \Rightarrow G(t) = c_3 e^{-\lambda_n t}$$

$$\Rightarrow u(x,t) = c \cos\left(\left(n\pi + \frac{\pi}{2}\right)x\right) e^{-\left(n\pi + \frac{\pi}{2}\right)^2 t}$$

applying I.V. $u(x,0) = \cos\left(\frac{\pi x}{2}\right) \Rightarrow n=0, c=1$

$$\text{so } u(x,t) = \cos\left(\frac{\pi x}{2}\right) e^{-\left(\frac{\pi}{2}\right)^2 t}$$

$$1.b.) \Rightarrow f'(x) = a f(x) + b f(x+\Delta x) + c f(x+2\Delta x)$$

$$f(x+\Delta x) = f(x) + \Delta x f'(x) + \frac{\Delta x^2}{2} f''(x) + O(\Delta x^3)$$

$$f(x+2\Delta x) = f(x) + 2\Delta x f'(x) + 2\Delta x^2 f''(x) + O(\Delta x^3)$$

$$\begin{aligned} \text{So } f'(x) &= a f(x) + b \left[f(x) + \Delta x f'(x) + \frac{\Delta x^2}{2} f''(x) + O(\Delta x^3) \right] + \\ &\quad + c \left[f(x) + 2\Delta x f'(x) + 2\Delta x^2 f''(x) + O(\Delta x^3) \right] \\ &= \underbrace{[a+b+c]}_{=0} f(x) + \underbrace{\Delta x [b+2c]}_{=1} f'(x) + \underbrace{\Delta x^2 \left[\frac{b}{2} + 2c \right]}_{=0} f''(x) + O(\Delta x^3) \end{aligned}$$

$$\text{So } a+b+c=0 \quad b+2c=1 \quad b+4c=0$$

$$\Rightarrow b = -4c \quad -4c + 2c = 1 \Rightarrow c = -\frac{1}{2}$$

$$\Rightarrow b = 2$$

$$\text{So } a = -3/2$$

$$\Rightarrow f'(x) = \frac{-\frac{3}{2}f(x) + 2f(x+\Delta x) - \frac{1}{2}f(x+2\Delta x)}{\Delta x} + O(\Delta x^2)$$

$$1.c.) \text{ so for the heat eqn. } r \leq \frac{1}{2} \text{ where } r = \frac{\nu \Delta t}{\Delta x^2}$$

$$\Rightarrow \Delta t \leq \frac{\Delta x^2}{2\nu} = \frac{0.01^2}{2(1/6)} = \boxed{0.03}$$

1.d.) This method is not very stable.

After approximately 0.5s, it begins to diverge.

3.) Show BTCS is P.C. for heat eqn.

□ given $V_t = \gamma V_{xx}$ w/ $\gamma > 0$, let $x \in \mathbb{R}$, $t > 0$

we partition x, t as $x_k = k\Delta x$ & $t_n = n\Delta t$ w/ $\Delta x, \Delta t > 0$

$$\text{BTCS: } \frac{u_k^{n+1} - u_k^n}{\Delta t} = \gamma \left(\frac{u_{k+1}^{n+1} - 2u_k^{n+1} + u_{k-1}^{n+1}}{\Delta x^2} \right)$$

$$\text{so let } \tau_k^n = \frac{v_k^{n+1} - v_k^n}{\Delta t} = \gamma \left(\frac{v_{k+1}^{n+1} - 2v_k^{n+1} + v_{k-1}^{n+1}}{\Delta x^2} \right)$$

expand v_k^n in time, v_{k+1}^{n+1} & v_{k-1}^{n+1} in space

$$v_k^n = v_k^{n+1} - \Delta t (v_t)_k^{n+1} + \frac{\Delta t^2}{2} (v_{tt})_k^{n+1} + O(\Delta t^3)$$

$$v_{k+1}^{n+1} = v_k^{n+1} + \Delta x (v_x)_k^{n+1} + \frac{\Delta x^2}{2} (v_{xx})_k^{n+1} + \frac{\Delta x^3}{3!} (v_{xxx})_k^{n+1} + \frac{\Delta x^4}{4!} (v_{xxxx})_k^{n+1} + O(\Delta x^5)$$

$$v_{k-1}^{n+1} = v_k^{n+1} - \Delta x (v_x)_k^{n+1} + \frac{\Delta x^2}{2} (v_{xx})_k^{n+1} - \frac{\Delta x^3}{3!} (v_{xxx})_k^{n+1} + \frac{\Delta x^4}{4!} (v_{xxxx})_k^{n+1} + O(\Delta x^5)$$

$$\text{so } \tau_k^n = \frac{v_k^{n+1}}{\Delta t} - \left(\frac{v_k^{n+1}}{\Delta t} - (v_t)_k^{n+1} + \frac{\Delta t}{2} (v_{tt})_k^{n+1} + O(\Delta t^2) \right) =$$

$$= \frac{\gamma}{\Delta x^2} \left(v_k^{n+1} + \frac{\Delta x^2}{2} (v_{xx})_k^{n+1} + \frac{\Delta x^4}{4!} (v_{xxxx})_k^{n+1} - 2v_k^{n+1} + v_k^{n+1} + \frac{\Delta x^2}{2} (v_{xx})_k^{n+1} + \frac{\Delta x^4}{4!} (v_{xxxx})_k^{n+1} + O(\Delta x^5) \right)$$

$$\tau_k^n = \underbrace{(v_t)_k^{n+1} - \gamma (v_{xx})_k^{n+1}}_{\text{PDE} = 0} + O(\Delta t) + O(\Delta x^2)$$

$$\tau_k^n = O(\Delta t) + O(\Delta x^2) \rightarrow 0 \text{ as } \Delta x, \Delta t \rightarrow 0$$

4.) Show FTCS is not P.C. w/ first-order of NBC
 for $V_t = V_{xx}$, $x \in (0,1)$, $t > 0$; $u(x,0) = f(x)$, $x \in [0,1]$
 $V_x(0,t) = V(1,t) = 0$

□ let $x_k = k\Delta x$ & $t_n = n\Delta t$ be partitions of x & t

$$\text{FTCS: } \frac{u_k^{n+1} - u_k^n}{\Delta t} = \frac{u_{k+1}^n - 2u_k^n + u_{k-1}^n}{\Delta x^2}$$

$$\text{BCs: } \frac{u_0^n - u_{-1}^n}{\Delta x} = 0 \Rightarrow u_{-1}^n = u_0^n$$

$$\text{Let } \tau_k^n = \frac{v_k^{n+1} - v_k^n}{\Delta t} - \left(\frac{v_{k+1}^n - 2v_k^n + v_{k-1}^n}{\Delta x^2} \right)$$

But we only care about Boundaries

$$\text{so examine } \tau_0^n = \frac{v_0^{n+1} - v_0^n}{\Delta t} - \left(\frac{v_1^n - 2v_0^n + v_{-1}^n}{\Delta x^2} \right)$$

$$= \frac{v_0^{n+1} - v_0^n}{\Delta t} - \left(\frac{v_1^n - v_0^n}{\Delta x^2} \right)$$

expand v_0^{n+1} in time & v_1^n in space

$$v_0^{n+1} = v_0^n + (v_t)_0^n \Delta t + \frac{(v_{tt})_0^n \Delta t^2}{2!} + O(\Delta t^3)$$

$$v_1^n = v_0^n + (v_x)_0^n \Delta x + \frac{(v_{xx})_0^n \Delta x^2}{2!} + \frac{(v_{xxx})_0^n \Delta x^3}{3!} + O(\Delta x^4)$$

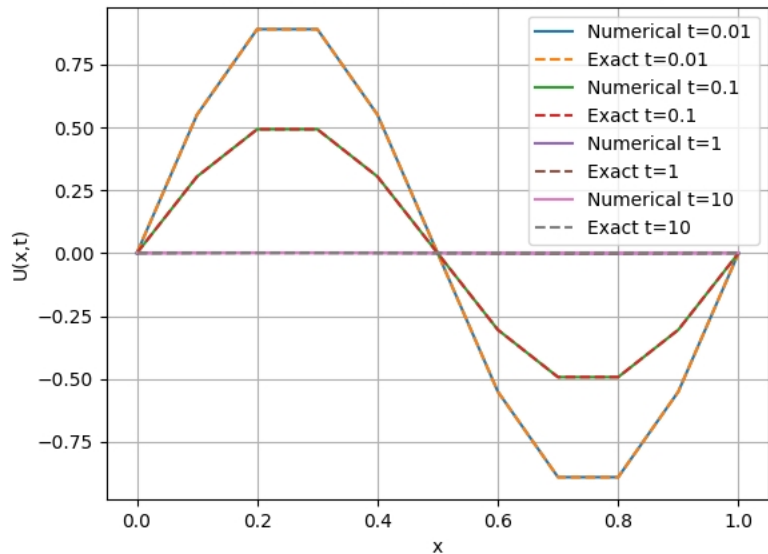
$$\text{so } \tau_0^n = \frac{v_0^n + (v_t)_0^n \Delta t + \frac{(v_{tt})_0^n \Delta t^2}{2!} - v_0^n}{\Delta t} - \frac{1}{\Delta x^2} \left(v_0^n + (v_x)_0^n \Delta x + \frac{(v_{xx})_0^n \Delta x^2}{2!} + \frac{(v_{xxx})_0^n \Delta x^3}{3!} + O(\Delta x^4) - v_0^n \right)$$

$$\begin{aligned}
 4. \text{ cont.) } \tau_0^{\wedge} &= (V_t)_0^{\wedge} + O(\Delta t) - \underbrace{\frac{(V_x)_0^{\wedge}}{\Delta x}}_{=0} \rightarrow \frac{(V_{xx})_0^{\wedge}}{2} + O(\Delta x) \\
 &= \underbrace{(V_t)_0^{\wedge} - (V_{xx})_0^{\wedge}}_{PDE = 0} + \frac{(V_{xx})_0^{\wedge}}{2} + O(\Delta t) + O(\Delta x)
 \end{aligned}$$

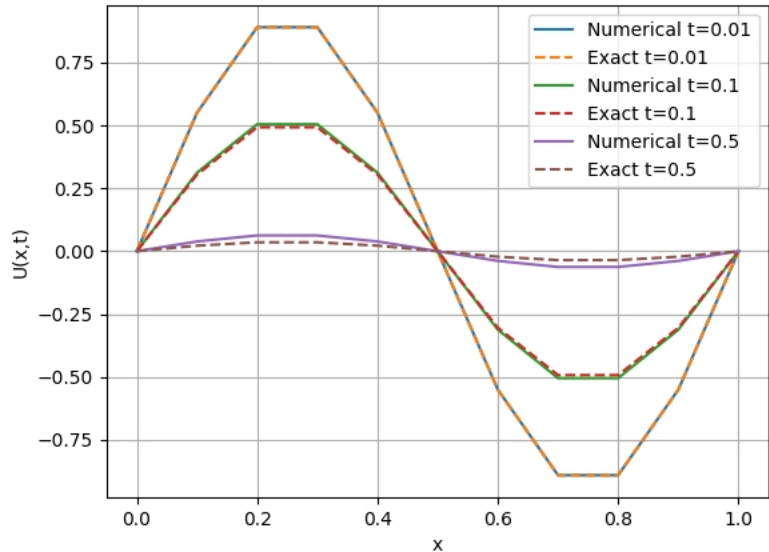
$$\tau_0^{\wedge} = \frac{(V_{xx})_0^{\wedge}}{2} + O(\Delta x) + O(\Delta t)$$

$$\tau_0^{\wedge} \not\rightarrow 0 \text{ as } \Delta x, \Delta t \rightarrow 0$$

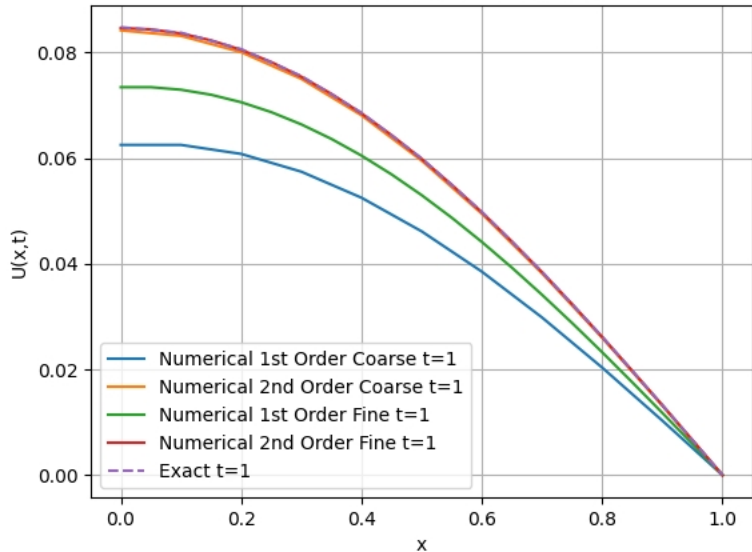
Problem 1.c) Numerical vs Exact Solutions



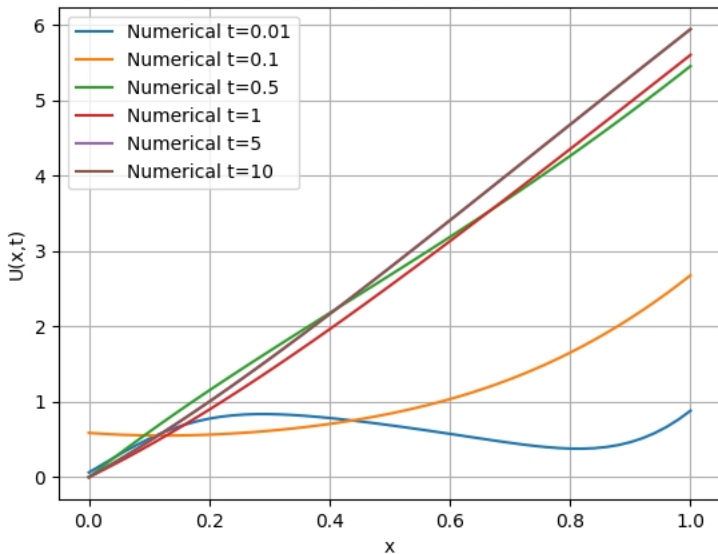
Problem 1.d) Numerical vs Exact Solutions



Problem 1.e) Numerical vs Exact Solutions



Problem 2: Numerical vs Exact Solutions



```
1 # Code for HW 1 for FA25 Advanced Numerical Analysis
2
3 # All code is original and written by me, Tommy Stell, for Advanced Numerical
  Analysis
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 # Problem 1
9
10 # I:  $U_t = v U_{xx}$ ,  $0 < x < 1$ ,  $t > 0$ 
11 #  $U(0,t) = 0$ ,  $U(1,t) = 0$ ,  $t \geq 0$ 
12 #  $U(x,0) = \sin(2\pi x)$ ,  $0 \leq x \leq 1$ 
13
14 # II:  $U_t = U_{xx}$ ,  $0 < x < 1$ ,  $t > 0$ 
15 #  $U(x,0) = \cos(\pi x/2)$ ,  $0 \leq x \leq 1$ 
16 #  $U_x(0,t) = U(1,t) = 0$ ,  $t \geq 0$ 
17
18 # 1.a)
19
20 exact_sol_1_I = lambda x, t, v: np.sin(2 * np.pi * x) * np.exp(
21     -v * (2 * np.pi) ** 2 * t
22 )
23 exact_sol_1_II = lambda x, t: np.cos(np.pi * x / 2) * np.exp(-((np.pi / 2) **
24     2) * t)
25
26 # 1.c)
27
28 v_1c = 1 / 6
29 del_x_1c = 1 / 10
30 del_t_1c = 0.01
31
32 sample_t_1c = [0.01, 0.1, 1, 10]
33
34 x_1c = np.arange(0, 1 + del_x_1c, del_x_1c)
35 num_x_1c = len(x_1c)
36 num_t_1c = int(10 / del_t_1c)
37
38 u_initial_1c = np.sin(2 * np.pi * x_1c)
39
40 sols_num = {}
41 sols_exact = {}
42
43 u = u_initial_1c.copy()
44
45 for i in range(1, num_t_1c + 1):
46     u_temp = u.copy()
47     for j in range(1, num_x_1c - 1):
48         u[j] = u_temp[j] + v_1c * del_t_1c / del_x_1c**2 * (
```

```
48         u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
49     )
50
51     t_now = i * del_t_1c
52     if t_now in sample_t_1c:
53         sols_num[t_now] = u.copy()
54         sols_exact[t_now] = exact_sol_1_I(x_1c, t_now, v_1c)
55
56 plt.figure()
57 for t in sample_t_1c:
58     plt.plot(x_1c, sols_num[t], label=f"Numerical t={t}")
59     plt.plot(x_1c, sols_exact[t], "--", label=f"Exact t={t}")
60
61 plt.title("Problem 1.c) Numerical vs Exact Solutions")
62 plt.xlabel("x")
63 plt.ylabel("U(x,t)")
64 plt.legend()
65 plt.grid()
66 plt.savefig("HW1_Problem1c_Solutions.png")
67
68 # by the stability theory, we need  $v \cdot \text{del\_t} / \text{del\_x}^2 \leq 1/2$ 
69 stability_ratio_1c = v_1c * del_t_1c / del_x_1c**2
70 print(f"Stability ratio for Problem 1.c): {stability_ratio_1c}")
71 print(f"The maximum allowable del_t for stability is: {0.5 * del_x_1c**2 / v_1c}")
72
73
74 # 1.d)
75
76 v_1d = 1 / 6
77 del_x_1d = 0.1
78 del_t_1d = 0.01
79 # del_t_1d = del_t_1d**2/(6*v_1d)
80
81 sample_t_1d = [0.01, 0.1, 0.5]
82
83 x_1d = np.arange(0, 1 + del_x_1d, del_x_1d)
84 num_x_1d = len(x_1d)
85 num_t_1d = int(1 / del_t_1d)
86
87 u_initial_1d = np.sin(2 * np.pi * x_1d)
88
89 u = u_initial_1d.copy()
90 u_old = u_initial_1d.copy()
91
92 sols_num_1d = {}
93 sols_exact_1d = {}
94
95 for i in range(1, num_t_1d + 1):
96     u_temp = u.copy()
97     if i == 1:
```

```
98         for j in range(1, num_x_1d - 1):
99             u[j] = u_temp[j] + v_1d * del_t_1d / del_x_1d**2 * (
100                 u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
101             )
102     else:
103         for j in range(1, num_x_1d - 1):
104             u[j] = u_old[j] + 2 * v_1d * del_t_1d / del_x_1d**2 * (
105                 u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
106             )
107     u_old = u_temp.copy()
108
109     t_now = i * del_t_1d
110     if t_now in sample_t_1d:
111         sols_num_1d[t_now] = u.copy()
112         sols_exact_1d[t_now] = exact_sol_1_I(x_1d, t_now, v_1d)
113
114 plt.figure()
115 for t in sample_t_1d:
116     plt.plot(x_1d, sols_num_1d[t], label=f"Numerical t={t}")
117     plt.plot(x_1d, sols_exact_1d[t], "--", label=f"Exact t={t}")
118
119 plt.title("Problem 1.d) Numerical vs Exact Solutions")
120 plt.xlabel("x")
121 plt.ylabel("U(x,t)")
122 plt.legend()
123 plt.grid()
124 plt.savefig("HW1_Problem1d_Solutions.png")
125
126 # 1.e)
127
128 del_x_1e_a = 0.1
129 del_t_1e_a = 0.004
130
131 del_x_1e_b = 0.05
132 del_t_1e_b = 0.001
133
134 sample_t_1e = [1]
135
136 x_1e_a = np.arange(0, 1 + del_x_1e_a, del_x_1e_a)
137 x_1e_b = np.arange(0, 1 + del_x_1e_b, del_x_1e_b)
138
139 num_x_1e_a = len(x_1e_a)
140 num_x_1e_b = len(x_1e_b)
141 num_t_1e_a = int(1 / del_t_1e_a)
142 num_t_1e_b = int(1 / del_t_1e_b)
143
144 u_initial_1e_a = np.cos(np.pi * x_1e_a / 2)
145 u_initial_1e_b = np.cos(np.pi * x_1e_b / 2)
146
147 u_a = u_initial_1e_a.copy()
148 u_b = u_initial_1e_b.copy()
```

```
149
150 sols_exact_1e = {}
151 sols_num_1e_a_first = {}
152 sols_num_1e_b_first = {}
153 sols_num_1e_a_second = {}
154 sols_num_1e_b_second = {}
155
156 # first order approximation of NBCs
157 for i in range(1, num_t_1e_a + 1):
158     u_temp = u_a.copy()
159     for j in range(1, num_x_1e_a - 1):
160         u_a[j] = u_temp[j] + del_t_1e_a / del_x_1e_a**2 * (
161             u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
162         )
163     # Neumann BCs
164     u_a[0] = u_a[1] # U_x(0,t) = 0
165     u_a[-1] = 0 # U(1,t) = 0
166     t_now = i * del_t_1e_a
167     if t_now in sample_t_1e:
168         sols_num_1e_a_first[t_now] = u_a.copy()
169         sols_exact_1e[t_now] = exact_sol_1_II(x_1e_a, t_now)
170
171 # second order approximation of NBCs
172 u_a = u_initial_1e_a.copy()
173 for i in range(1, num_t_1e_a + 1):
174     u_temp = u_a.copy()
175     for j in range(1, num_x_1e_a - 1):
176         u_a[j] = u_temp[j] + del_t_1e_a / del_x_1e_a**2 * (
177             u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
178         )
179     # Neumann BCs
180     u_a[0] = u_temp[0] + 2 * del_t_1e_a / del_x_1e_a**2 * (
181         u_temp[1] - u_temp[0]
182     ) # U_x(0,t) = 0
183     u_a[-1] = 0 # U(1,t) = 0
184     t_now = i * del_t_1e_a
185     if t_now in sample_t_1e:
186         sols_num_1e_a_second[t_now] = u_a.copy()
187
188 # first order refined
189 u_b = u_initial_1e_b.copy()
190 for i in range(1, num_t_1e_b + 1):
191     u_temp = u_b.copy()
192     for j in range(1, num_x_1e_b - 1):
193         u_b[j] = u_temp[j] + del_t_1e_b / del_x_1e_b**2 * (
194             u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
195         )
196     # Neumann BCs
197     u_b[0] = u_b[1] # U_x(0,t) = 0
198     u_b[-1] = 0 # U(1,t) = 0
199     t_now = i * del_t_1e_b
```

```

200     if t_now in sample_t_1e:
201         sols_num_1e_b_first[t_now] = u_b.copy()
202
203 # second order refined
204 u_b = u_initial_1e_b.copy()
205 for i in range(1, num_t_1e_b + 1):
206     u_temp = u_b.copy()
207     for j in range(1, num_x_1e_b - 1):
208         u_b[j] = u_temp[j] + del_t_1e_b / del_x_1e_b**2 * (
209             u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
210         )
211     # Neumann BCs
212     u_b[0] = u_temp[0] + 2 * del_t_1e_b / del_x_1e_b**2 * (
213         u_temp[1] - u_temp[0]
214     ) # U_x(0,t) = 0
215     u_b[-1] = 0 # U(1,t) = 0
216     t_now = i * del_t_1e_b
217     if t_now in sample_t_1e:
218         sols_num_1e_b_second[t_now] = u_b.copy()
219
220 plt.figure()
221 for t in sample_t_1e:
222     plt.plot(x_1e_a, sols_num_1e_a_first[t], label=f"Numerical 1st Order Coarse t={t}")
223     plt.plot(x_1e_a, sols_num_1e_a_second[t], label=f"Numerical 2nd Order Coarse t={t}")
224     plt.plot(x_1e_b, sols_num_1e_b_first[t], label=f"Numerical 1st Order Fine t={t}")
225     plt.plot(x_1e_b, sols_num_1e_b_second[t], label=f"Numerical 2nd Order Fine t={t}")
226     plt.plot(x_1e_a, sols_exact_1e[t], "--", label=f"Exact t={t}")
227
228 plt.title("Problem 1.e) Numerical vs Exact Solutions")
229 plt.xlabel("x")
230 plt.ylabel("U(x,t)")
231 plt.legend()
232 plt.grid()
233 plt.savefig("HW1_Problem1e_Solutions.png")
234
235
236 # problem 2
237
238 #  $U_t = U_{xx}$ ,  $0 < x < 1$ ,  $t > 0$ 
239 #  $U(x,0) = \cos(\pi x/2)$   $0 \leq x \leq 1$ 
240 #  $U_x(0,t) = \sin(2\pi t)$ ,  $U_x(1,t) = 2\pi$ ,  $t \geq 0$ 
241
242 del_x_2 = 0.01
243 del_t_2 = 0.00005
244
245 x_2 = np.arange(0, 1 + del_x_2, del_x_2)
246 num_x_2 = len(x_2)

```



```
247 num_t_2 = int(10 / del_t_2)
248
249 u_initial_2 = np.cos(np.pi * x_2) / 2
250
251 u = u_initial_2.copy()
252
253 sample_t_2 = [0.01, 0.1, 0.5, 1, 5, 10]
254 sols_num_2 = {}
255
256
257 # second order approximation for the BCs
258
259 for i in range(1, num_t_2 + 1):
260     u_temp = u.copy()
261     for j in range(1, num_x_2 - 1):
262         u[j] = u_temp[j] + del_t_2 / del_x_2**2 * (
263             u_temp[j + 1] - 2 * u_temp[j] + u_temp[j - 1]
264         )
265
266     # Apply boundary conditions
267     u[0] = np.sin(2*np.pi*i*del_t_2)
268
269     u[-1] = (
270         u_temp[-1]
271         + 2 * del_t_2 / del_x_2**2 * (u_temp[-2] - u_temp[-1])
272         + 2 * del_t_2 / del_x_2 * 2 * np.pi
273     )
274
275     t_now = i * del_t_2
276     if t_now in sample_t_2:
277         sols_num_2[t_now] = u.copy()
278
279 plt.figure()
280 for t in sample_t_2:
281     plt.plot(x_2, sols_num_2[t], label=f"Numerical t={t}")
282
283 plt.title("Problem 2: Numerical vs Exact Solutions")
284 plt.xlabel("x")
285 plt.ylabel("U(x,t)")
286 plt.legend()
287 plt.grid()
288 plt.savefig("HW1_Problem2_Solutions.png")
289
```